Measuring the Systems Cost of Heterogeneous KV Quantization

Shivam Patel (2025202030)
Suyash Pande (2025201063)
Divyanshu Jain (2025201059)

Domain: LLMs & Foundation Models
Sub-domains: KV Cache Optimization for Long-Context Models; Quantization & Low-Precision Foundation Models

---

1 Problem Statement and Motivation

Today's LLM serving systems store the KV cache in 4-bit, but they use the **same bit-width for every layer and every head**, and they use no calibration data. Research papers say we can do better by giving **different heads different bit-widths**, and they claim this costs nothing extra at inference time. We will build the kernel and measure the cost.

1.1 Why the KV cache is the bottleneck

When an LLM generates text, each new token must look at the entire KV cache. The cache grows with the context length and is read completely at every step. This makes long-context decoding limited by memory bandwidth, not by compute.

For Llama-3.1-8B (32 layers, 8 KV heads, head dimension 128, FP16), the cache costs:

$$2 \times 32 \times 8 \times 128 \times 2 = 131072 \text{ bytes} = 128 \text{ KB per token}$$

Compared against INT4 model weights of about 5.7 GB:

Setting	Weight traffic/step	KV traffic/step	KV share
32K context, batch 1	5.7 GB	4.0 GB	41%
32K context, batch 4	5.7 GB	16.0 GB	74%
32K context, batch 8	5.7 GB	32.0 GB	**85%**

Table 1: Memory traffic per decode step. Weights are shared across a batch; the KV cache is not.

Weight reads are shared across all requests in a batch. KV reads are not. On a 936 GB/s GPU, the first row alone means about 4.3 ms per token is spent just reading the KV cache. So reducing the number of bytes read is the only thing that helps.

1.2 Our question

> *Is mixed-precision KV actually free at inference time, or do uneven page sizes, per-head dispatch, warp divergence and unpacking work eat up the savings?*

Both answers are useful. If the cost is small, mixed precision should go into production engines. If the cost is large, then the papers are optimizing a number that does not translate to real hardware, and finding the point where it stops paying off becomes our contribution.

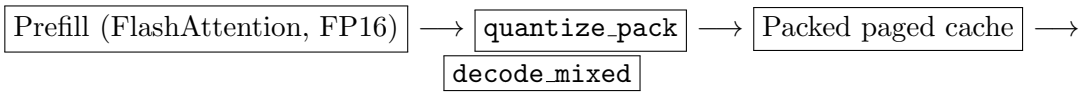
2 Proposed Approach

We will build a paged KV cache where the bit-width changes per layer and per head, and a fused CUDA decoding kernel that reads those packed values directly from GPU memory without ever building a full FP16 cache. Then we measure what the mixed precision costs.

2.1 Plan of Action

1. **Kernel experiments use synthetic data, not real models.** A decoding kernel reads a KV cache. It does not need the model weights that produced that cache. So we allocate KV buffers with exactly the shape of Llama-3.1-8B ($32 \times 8 \times 128$), fill them with random data of matching distribution, and benchmark against that. All bandwidth, roofline, coalescing and divergence results come from this setup. It loads no model, runs in seconds, and produces most of our findings.
2. **Development happens off the lab GPU.** Kaggle and Colab give free T4 (sm_75) sessions with weekly quotas much larger than our lab allocation. All compiling, correctness testing against PyTorch SDPA, and numerical debugging happens there. Lab GPU time is used only for final measurements on the target hardware.
3. **End-to-end quality tests use a small model.** Llama-3.2-3B-Instruct uses 112 KB per token and 6.4 GB of FP16 weights, so a full 128K-context FP16 baseline still fits in 24 GB. Qwen2.5-1.5B-Instruct (28 KB per token) is our second model family.

2.2 Three kernels



Prefill. Standard FlashAttention in FP16. Not our contribution.

`quantize_pack`**.** Compresses the FP16 output of prefill into our packed paged format. This is a one-time cost spread over thousands of decoding steps. We will profile it and show how quickly it is amortized.

`decode_mixed`**.** The main kernel. A Flash-Decoding style split-K design:

- Thread blocks split the KV sequence between them. Each computes a partial softmax state (m, l, acc), and a second kernel combines them. This is needed because batch-1 decoding with 8 KV heads would otherwise use only 8 thread blocks on an 82-SM GPU.

- Per-token asymmetric quantization with one scale and zero-point per page. This layout keeps 128-bit `uint4` loads coalesced.
- INT4 unpacking using the AWQ bit trick: OR the nibbles into the FP16 exponent pattern `0x6400` and subtract. This de-quantizes using `lop3` and `hsub2` instead of slower integer-to-float conversion.

2.3 The mixed-precision experiment

We compare two ways of handling different bit-widths, using profiler evidence:

Strategy	**How it works**	**Expected cost**
A: templated kernels	One kernel compiled per bit-width, chosen per head group	More kernel launches, no divergence
B: runtime branching	One kernel that reads a per-head bit-width at runtime	Fewer launches, divergent warps

Table 2: The two dispatch strategies we will compare.

Which one wins, and by how much, is our core measurement. We use `smsp__thread_inst_executed_per_inst_e` to measure divergence in B, and kernel launch counts from Nsight Systems to measure the overhead in A.

2.4 Choosing the bit-widths

We use a simple offline, forward-only method. We change one head's KV to a candidate bit-width, measure how much the loss increases on a small calibration set, and then greedily assign bits under an average-bit budget. For Llama-3.2-3B this is $28 \times 8 = 224$ forward passes, which takes under an hour at 3B scale.

We deliberately avoid gradient-based sensitivity, even though papers prefer it, because backward passes will not fit in memory alongside a long-context cache. This is the least novel part of the project. Our contribution is running the allocation in a real kernel, not inventing a better allocator.

3 Hardware and Software Platform

3.1 Target hardware

GPU	24–32 GB VRAM, compute capability ≥ 8.0 (Ampere or Ada)
Likely cards	RTX 3090 / A5000 / L40S / RTX 6000 Ada
Memory bandwidth	≥ 700 GB/s
Allocation	About 30 GPU-hours total across nine weeks

3.2 Software

- CUDA Toolkit 12.4 or later, with `nvcc` available to our user account (we compile custom kernels)
- NVIDIA driver 550 or later

- Nsight Compute (`ncu`) and Nsight Systems (`nsys`), 2024.3 or later
- PyTorch 2.4+, HuggingFace `transformers`, our kernels built as a `torch.utils.cpp_extension` module
- LMDeploy (TurboMind) and vLLM, used only as external baselines

3.3 Models

Model	KV/token	Role
Llama-3.1-8B shapes (synthetic)	128 KB	Kernel benchmarks, no weights loaded
Llama-3.2-3B-Instruct	112 KB	End-to-end quality and capacity, 128K context
Qwen2.5-1.5B-Instruct	28 KB	Second model family, INT4 sensitivity check

3.4 Permissions we may need

1. **Profiling counters.** `ncu` fails with `ERR_NVGPUCTRPERM` unless `NVreg_RestrictProfilingToAdminUsers=0` is set on the NVIDIA kernel module. Profiling evidence is 20% of the grade.
2. **Clock control.** `nvidia-smi -pm 1` and `nvidia-smi -lgc`. Without locked clocks, heating makes timings non-reproducible.
3. **Exclusive access during measurement windows.** Not needed for development, only for the booked sessions above, since sharing the GPU corrupts latency and power numbers.

4 Baseline

We use four baselines, each answering a different question. Our main comparison is against our own FP16 kernel.

Baseline	Question it answers	Role
HuggingFace FP16 `DynamicCache`	Is our output numerically correct?	Correctness reference
Our FP16 paged split-K kernel	What does quantization alone buy?	**Main comparison**
LMDeploy TurboMind int4 KV	How far are we from shipped 4-bit KV?	Production state of the art
KIVI-style simulated INT4	What quality is possible ignoring systems cost?	Quality upper bound

4.1 Correctness checks

Every kernel is validated before it is benchmarked. We compare outputs against PyTorch SDPA on identical inputs, with tolerances set per precision. For the FP16 path we require maximum absolute error below 10^{-3}. Quantized paths are compared against a PyTorch simulation of the same quantization scheme, not against FP16. If a kernel has not passed this check, we report no numbers from it.

5 Evaluation Metrics

5.1 Performance

Metric	Source	What it shows
DRAM bandwidth vs peak	`dram__throughput`	That we are bandwidth-bound, so bytes are the only lever
DRAM bytes read per step	`dram__bytes_read.sum`	That 4-bit storage really moves 4× fewer bytes
Sectors per request	`l1tex__average_t_sectors_per_request`	That our uneven layout keeps memory coalesced
Warp divergence	`smsp__thread_inst_executed_per_inst_executed`	The cost of runtime bit-width dispatch
Kernel launches per step	`nsys` timeline	The cost of templated kernels
Decode latency vs context	CUDA events	The headline performance curve

5.2 Capacity

Maximum number of concurrent sessions at a fixed context length on one 24 GB GPU, FP16 KV versus ours. This is our demonstration for the final presentation.

5.3 Quality

- Perplexity on WikiText-2 and PG19 at 4K, 8K, 16K and 32K context
- Needle-in-a-haystack retrieval accuracy at several depths and context lengths
- All plotted against **average bits**, not scheme name, so uniform and mixed allocations are compared at equal cost

RULER subsets are a stretch goal. Given our GPU budget, perplexity plus needle-in-a-haystack is what we commit to.

5.4 Energy

Energy per token, from NVML power sampling during decoding. This is cheap to collect alongside latency runs and connects the work to the course's energy-efficiency theme.

5.5 Measurement protocol

Because experimental rigour is graded and our GPU time is limited, every number we report follows a fixed procedure:

1. Persistence mode on, clocks locked with `nvidia-smi -lgc` where allowed
2. 20 warmup iterations discarded, then 100 timed iterations
3. Median reported with interquartile range, never a single run or a bare mean
4. Run-to-run variance reported next to every speedup claim
5. If clocks cannot be locked, we report the variance and state the limitation

6 Expected Contribution

1. **A measurement of what mixed-precision KV actually costs.** The first real measurement of per-head bit allocation inside a fused kernel, against the literature's claim that it is free. Reported as a breakdown: dispatch overhead, divergence, coalescing loss, and unpacking instructions.
2. **A fused mixed-precision paged decoding kernel.** Open-sourced, reading INT4, INT8 and FP16 KV directly from a paged buffer with in-register de-quantization, supporting per-head bit-widths. Useful on its own, whatever our findings.
3. **A comparison of dispatch strategies.** Templated kernels versus runtime branching, with profiler evidence for which wins and at what granularity the answer flips.
4. **A crossover analysis.** The context length and batch size below which INT4 unpacking costs more than the bandwidth it saves. Practitioners currently have no guidance on when 4-bit KV stops being worth it.

7 Expected Outcome

Expected technical result

We expect to show that mixed-precision KV is *not* free at inference time, and to quantify by how much. Our working hypothesis is that template specialization (strategy A) beats runtime branching (strategy B) at coarse granularity, but that the launch overhead of A grows faster than the divergence cost of B as the number of distinct bit-widths rises, giving a crossover we can locate.

Concretely we expect to deliver:

- A working fused kernel reading INT4, INT8 and FP16 KV directly from a paged buffer
- 3–4× reduction in KV memory, letting a single 24 GB GPU hold several times more concurrent long-context sessions than FP16
- A measured cost of heterogeneity, reported as a breakdown rather than a single number
- The crossover context length below which 4-bit KV stops paying off

A negative result remains a full result. If mixed precision costs more than it saves, that finding redirects future work toward uniform schemes with better quantizers, which is more useful than another simulated perplexity table.

How success will be measured

Baselines. Four, each answering a different question:

The main comparison is deliberately against our own FP16 kernel. This keeps structure, tiling, launch configuration and author constant, so any measured difference comes from precision alone rather than from kernel-writing skill. We expect to reach 60–75% of TurboMind's throughput and will report that gap honestly with profiler evidence; no claim in this project depends on closing it.

Performance metrics.

Baseline	Question it answers	Role
HuggingFace FP16 `DynamicCache`	Is our output numerically correct?	Correctness reference
Our FP16 paged split-K kernel	What does quantization alone buy?	**Main comparison**
LMDeploy TurboMind int4 KV	How far are we from shipped 4-bit KV?	Production state of the art
KIVI-style simulated INT4	What quality is possible ignoring systems cost?	Quality upper bound

Metric	Source	What it shows
DRAM bandwidth vs peak	`dram__throughput`	That we are bandwidth-bound, so bytes are the only lever
DRAM bytes read per step	`dram__bytes_read.sum`	That 4-bit storage really moves $4\times$ fewer bytes
Kernel launches per step	`nsys` timeline	The cost of templated kernels
Decode latency vs context	CUDA events	The headline performance curve
Energy per token	NVML power sampling	Energy efficiency of each precision

Capacity. Maximum concurrent sessions at fixed context on one 24 GB GPU, FP16 KV versus ours. This is our demonstration for the final evaluation.

Quality. Perplexity on WikiText-2 and PG19 at 4K, 8K, 16K and 32K context, plus needle-in-a-haystack retrieval accuracy at several depths. All plotted against **average bits** rather than scheme name, so uniform and mixed allocations are compared at equal cost. RULER subsets are a stretch goal; perplexity plus needle-in-a-haystack is our committed scope.

Correctness gate. Every kernel is validated before it is benchmarked, against PyTorch SDPA on identical inputs. The FP16 path must show maximum absolute error below 10^{-3}; quantized paths are compared against a PyTorch simulation of the same scheme, not against FP16. A kernel that has not passed produces no numbers we report.

Measurement protocol. Because rigour is graded and GPU time is scarce, every reported number follows a fixed procedure: persistence mode on and clocks locked where permitted; 20 warmup iterations discarded, then 100 timed iterations; median reported with interquartile range, never a single run or a bare mean; run-to-run variance reported alongside every speedup claim.

8 Project Timeline

8.1 Team split

Each member owns a part that can be assessed independently.

The synthetic benchmark harness is built together in week 1 and used by all three of us, since it is the shared measurement base.

8.2 Critical path

Synthetic harness (wk 1) $\longrightarrow$ FP16 kernel (wk 3) $\longrightarrow$ Roofline and profiling (wk 4) $\longrightarrow$ **Eval 1** (wk 5)

FP16 kernel (wk 3) $\longrightarrow$ INT4 fused (wk 6) $\longrightarrow$ **Eval 2** (wk 7)

INT4 fused (wk 6) $\longrightarrow$ Mixed precision (wk 8) $\longrightarrow$ **Final** (wk 9)

Wk	Activity	Hrs	Milestone
1	Reading (KIVI, KVQuant, PagedAttention, FlashDecoding). Verify `ncu`, clock locking, VRAM. Build synthetic harness	2	Blockers cleared
2	Finalize proposal. Measure KV vs weight traffic on HuggingFace and TurboMind	2	Motivating numbers are our own
3	FP16 paged split-K kernel: write on Kaggle, validate against SDPA, measure on lab GPU	4	Tier 1 kernel correct
4	Establish roofline. Collect Nsight Compute metrics. Write up bottleneck	3	Bandwidth-bound claim proven
5	**Evaluation 1** — baseline, measurements, bottleneck, optimization plan	2	20% secured
6	`quantize_pack` kernel. INT4 packing and fused unpacking. Calibration sweep	5	Tier 2 kernel working
7	**Evaluation 2** — end-to-end generation, baseline comparison, first quality numbers	4	25% secured
8	Per-head mixed precision. Strategy A vs B. Second model family. Full sweeps	5	Tier 3, the differentiator
9	**Final Evaluation** — capacity demo, ablations, report	3	Complete

The FP16 kernel in week 3 is our single point of failure, since everything after it depends on it. We therefore give it two weeks of slack before Evaluation 1, and develop it on free Kaggle GPUs where we can iterate without limits.